Practicla No : 04
Name : Kiran Dadarao Janjal
Roll No : 13216
Batch: B1
Program :

```
import java.util.*;

class Process {     int pid; //
Process ID     int arrival; // Arrival
Time     int burst; // Burst Time
int priority; // Priority     int
remaining; // Remaining Time     int
completion; // Completion Time     int
waiting; // Waiting Time     int
turnaround; // Turnaround Time

    Process(int pid, int arrival, int burst, int priority) {
this.pid = pid;         this.arrival = arrival;
this.burst = burst;         this.priority = priority;
this.remaining = burst;
    }
}

public class SchedulingAlgorithms {

    // ---------- FCFS ----------
    static void fcfs(List<Process> processes) {
        System.out.println("\n--- First Come First Serve (FCFS) ---");
processes.sort(Comparator.comparingInt(p -> p.arrival));

        int time = 0;
        for (Process p : processes) {
if (time < p.arrival)
time = p.arrival;             time +=
p.burst;
            p.completion = time;
            p.turnaround = p.completion - p.arrival;
            p.waiting = p.turnaround - p.burst;
}
        printTable(processes);
    }
```

```
    // ---------- SJF Preemptive (SRTF) ----------
static void sjfPreemptive(List<Process> processes) {
        System.out.println("\n--- Shortest Job First (Preemptive -
SRTF) ---");

        int n = processes.size();
int time = 0, completed = 0;

        while (completed < n) {
Process shortest = null;             for
(Process p : processes) {
                if (p.arrival <= time && p.remaining > 0) {
if (shortest == null || p.remaining < shortest.remaining) {
                        shortest = p;
                    }
                }
            }

            if (shortest == null) {
time++;                 continue;
            }

            shortest.remaining--;
            time++;

            if (shortest.remaining == 0) {
shortest.completion = time;
                shortest.turnaround = shortest.completion -
shortest.arrival;
                shortest.waiting = shortest.turnaround -
shortest.burst;
                completed++;
            }
}
        printTable(processes);
    }

    // ---------- Priority (Non-Preemptive) ----------     static void
priorityNonPreemptive(List<Process> processes) {
System.out.println("\n--- Priority Scheduling (Non-Preemptive)
---");
        int time = 0, completed = 0;
int n = processes.size();         while
```

```
(completed < n) {             Process
highest = null;             for (Process
p : processes) {
                if (p.arrival <= time && p.remaining > 0) {
if (highest == null || p.priority < highest.priority) {
                        highest = p;
                    }
                }
            }

            if (highest == null) {
time++;                 continue;
            }

            time += highest.burst;
highest.remaining = 0;
highest.completion = time;
            highest.turnaround = highest.completion - highest.arrival;
highest.waiting = highest.turnaround - highest.burst;
            completed++;
        }
        printTable(processes);
    }

    // ---------- Round Robin (Preemptive) ----------     static
void roundRobin(List<Process> processes, int quantum) {
        System.out.println("\n--- Round Robin (Preemptive) ---");

        Queue<Process> queue = new LinkedList<>();
int time = 0, completed = 0, n = processes.size();

        processes.sort(Comparator.comparingInt(p -> p.arrival));
int i = 0;

        while (completed < n) {
            while (i < n && processes.get(i).arrival <= time) {
queue.add(processes.get(i));                    i++;
            }

            if (queue.isEmpty()) {
time++;                 continue;
            }
```

```
            Process p = queue.poll();
            int exec = Math.min(quantum, p.remaining);
            p.remaining -= exec;
time += exec;

            while (i < n && processes.get(i).arrival <= time) {
queue.add(processes.get(i));                  i++;
            }

            if (p.remaining > 0) {
queue.add(p);
            } else {
                p.completion = time;
                p.turnaround = p.completion - p.arrival;
                p.waiting = p.turnaround - p.burst;
completed++;
            }
}
        printTable(processes);
    }

    // ---------- Utility ----------
    static void printTable(List<Process> processes) {
System.out.println("PID\tAT\tBT\tPR\tCT\tTAT\tWT");          for
(Process p : processes) {
            System.out.println(p.pid + "\t" + p.arrival + "\t" +
p.burst + "\t" + p.priority
                    + "\t" + p.completion + "\t" + p.turnaround + "\t"
+ p.waiting);
        }
    }
    public static void main(String[] args) {
List<Process> processes = new ArrayList<>();
processes.add(new Process(1, 0, 5, 2));
processes.add(new Process(2, 1, 3, 1));
processes.add(new Process(3, 2, 8, 4));
processes.add(new Process(4, 3, 6, 3));

        // Run all algorithms separately with fresh process copies
fcfs(cloneProcesses(processes));
sjfPreemptive(cloneProcesses(processes));
priorityNonPreemptive(cloneProcesses(processes));
roundRobin(cloneProcesses(processes), 2);
```

```
    }

    // Clone list so each algorithm gets fresh burst/remaining times
static List<Process> cloneProcesses(List<Process> processes) {
        List<Process> newList = new ArrayList<>();
for (Process p : processes) {
            newList.add(new Process(p.pid, p.arrival, p.burst,
p.priority));
}
        return newList;
    }
}
```

OUTPUT :

```
--- First Come First Serve (FCFS) ---
PID     AT      BT      PR      CT      TAT     WT
1       0       5       2       5       5       0
2       1       3       1       8       7       4
3       2       8       4       16      14      6
4       3       6       3       22      19      13
Average Waiting Time: 5.75
Average Turnaround Time: 11.25
Gantt Chart:
|0 P1 |5 P2 |8 P3 |16 P4 |22|


--- Shortest Job First (Preemptive – SRTF) ---
PID     AT      BT      PR      CT      TAT     WT
1       0       5       2       8       8       3
2       1       3       1       4       3       0
3       2       8       4       22      20      12
4       3       6       3       14      11      5
Average Waiting Time: 5.00
Average Turnaround Time: 10.50
Gantt Chart:
|0 P1 |1 P2 |2 P2 |3 P2 |4 P1 |5 P1 |6 P1 |7 P1 |8 P4 |9 P4 |10 P4 |11 P4 |12 P4
|13 P4 |14 P3 |15 P3 |16 P3 |17 P3 |18 P3 |19 P3 |20 P3 |21 P3 |22|


--- Priority Scheduling (Non-Preemptive) ---
PID     AT      BT      PR      CT      TAT     WT
1       0       5       2       5       5       0
2       1       3       1       8       7       4
3       2       8       4       22      20      12
4       3       6       3       14      11      5
Average Waiting Time: 5.25
Average Turnaround Time: 10.75
Gantt Chart:
|0 P1 |5 P2 |8 P4 |14 P3 |22|


--- Round Robin (Preemptive) ---
```

```
PID     AT      BT      PR      CT      TAT     WT
1       0       5       2       14      14      9
2       1       3       1       11      10      7
3       2       8       4       22      20      12
4       3       6       3       20      17      11
Average Waiting Time: 9.75
Average Turnaround Time: 15.25
Gantt Chart:
|0 P1 |2 P2 |4 P3 |6 P1 |8 P4 |10 P2 |11 P3 |13 P1 |14 P4 |16 P3 |18 P4 |20 P3
|22|


[Program finished]
```